Git Assignment – NucleusTeq Training

Name: Palak Meena

Date: 11 March 2026

Git Repository: https://github.com/Palakmeena/nucleusteq-training

Introduction

This assignment was done as part of understanding how Git actually works in software development projects. The main goal was to get comfortable with the basic Git commands that developers use daily, things like setting up a repo, working with branches, merging changes, and keeping track of what has been done through commit history.

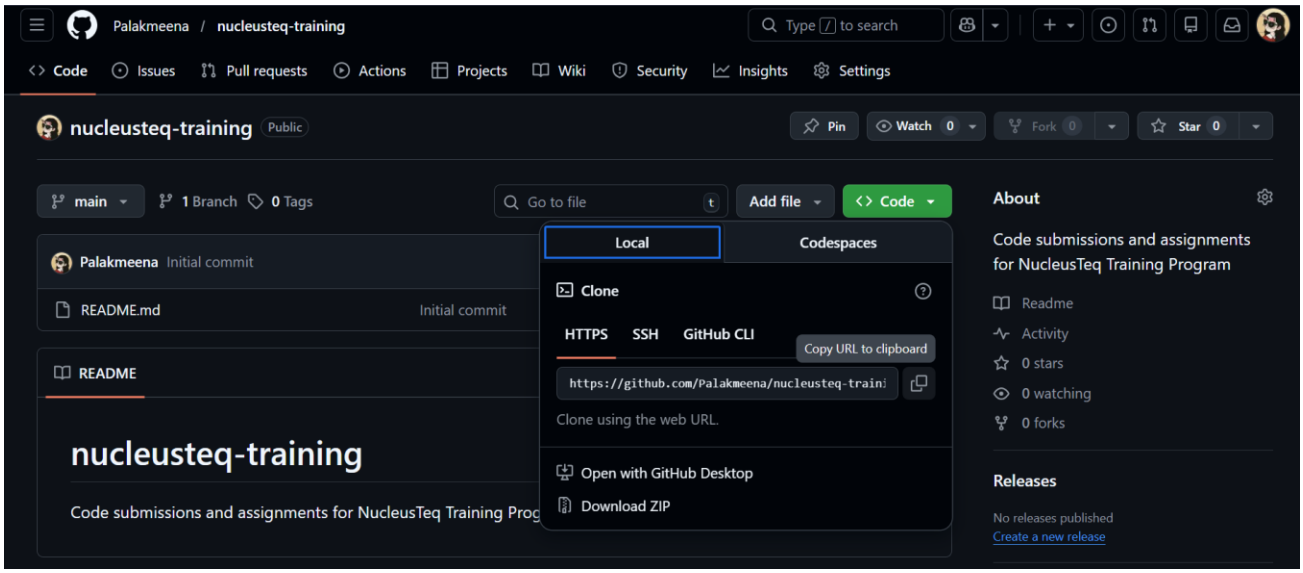
I also practised using revert and reset commands, which are really useful when something goes wrong and you need to undo changes. All of this was done using Git Bash along with Visual Studio Code.

Step 1: Create a New Repository

To get started, I first created a new repository called nucleusteq-training directly on GitHub. After that, I copied the repository URL and cloned it to my local system using the following command,

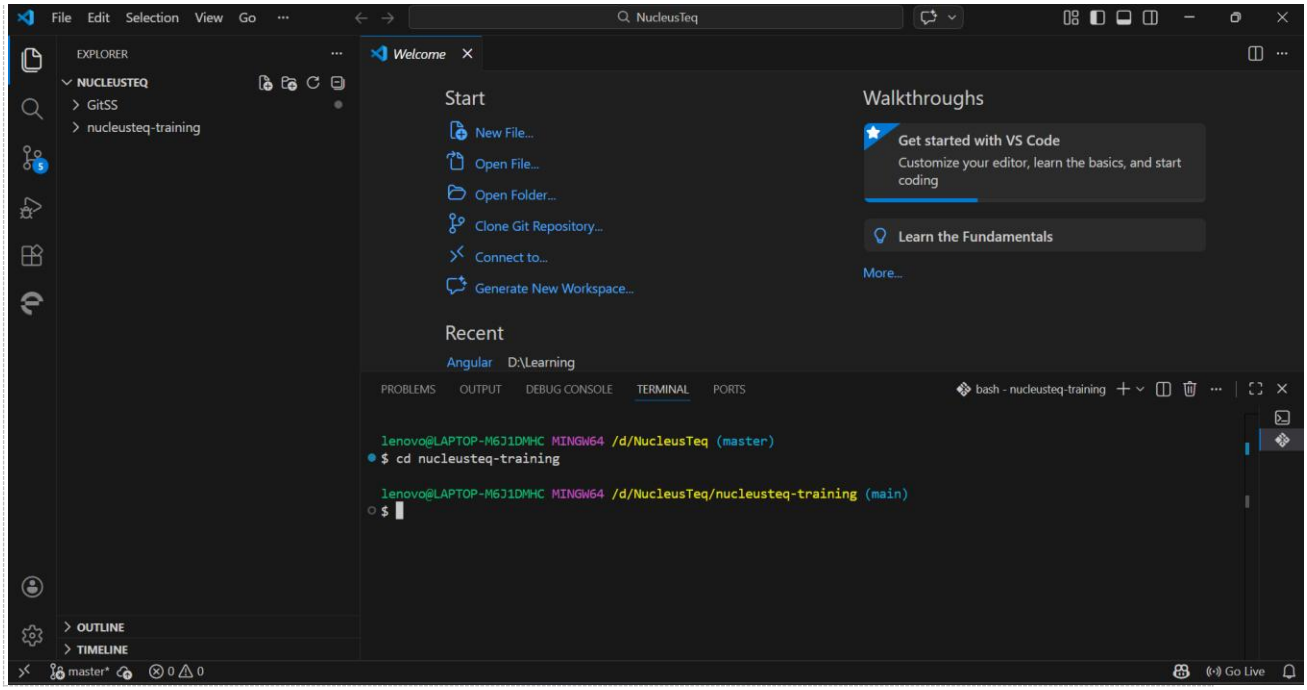
Command used:

git clone https://github.com/Palakmeena/nucleusteq-training



This created a local copy of the repository on my machine with the remote already connected. After cloning, I moved into the project folder:

```
cd nucleusteq-training
```



Step 2: Create a New File

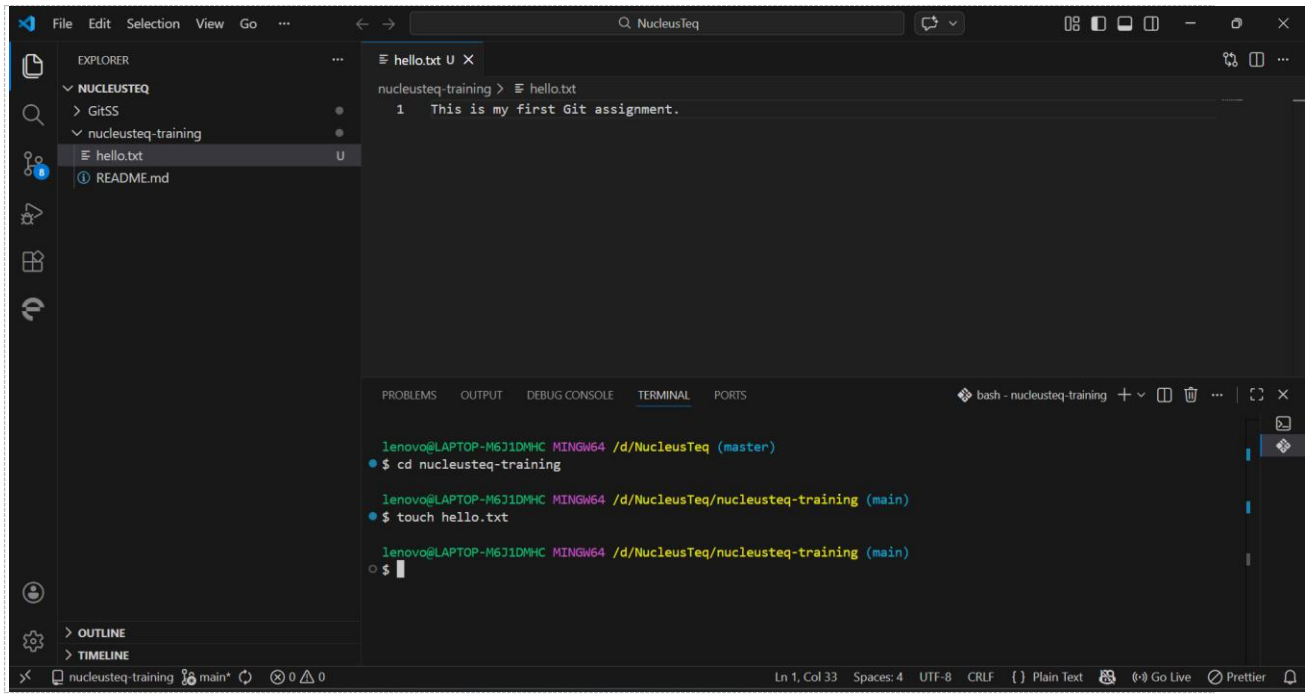
Next, I created a file named hello.txt inside the repository folder. This was the first file to be tracked by Git.

Command used:

```
touch hello.txt
```

After creating the file, I added it to the staging area and made the first commit:

```
git add hello.txt
git commit -m "Initial commit"
```



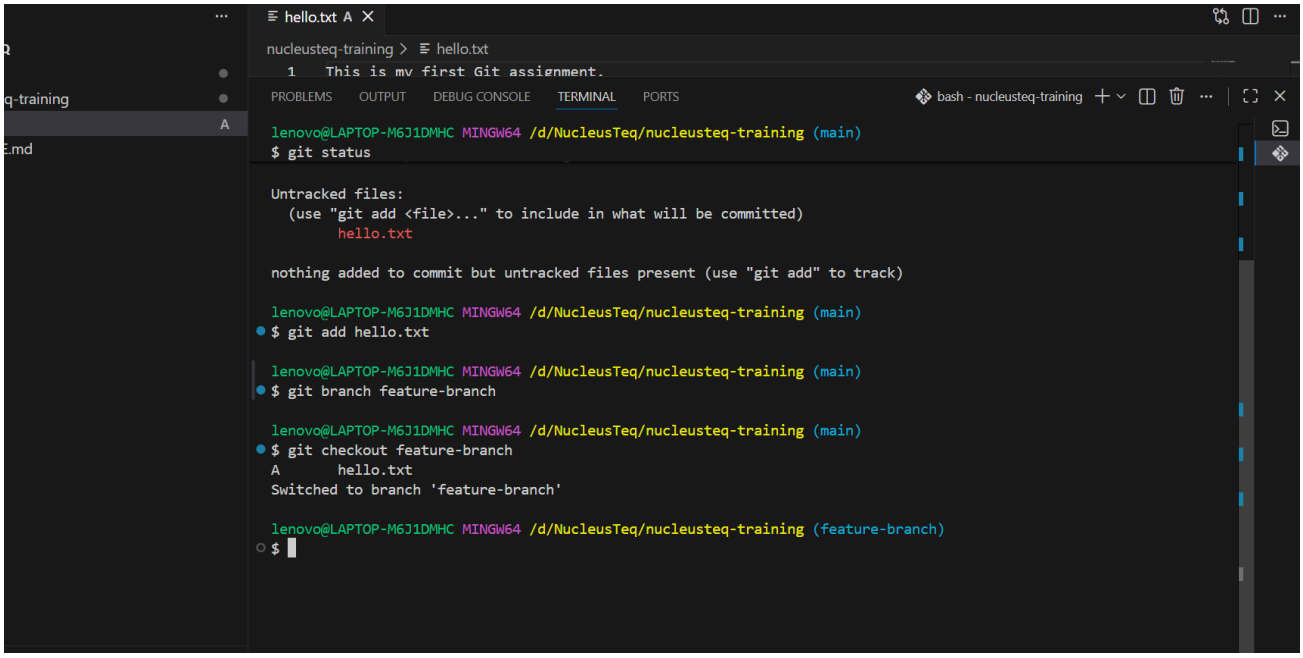
Step 3: Create a New Branch

I created a new branch called feature-branch. Working in a separate branch is a good practice because it keeps the main branch clean while you are still working on something.

Commands used:

```
git branch feature-branch
git checkout feature-branch
```

The first command creates the branch and the second one switches over to it. So now I am working inside feature-branch instead of main.

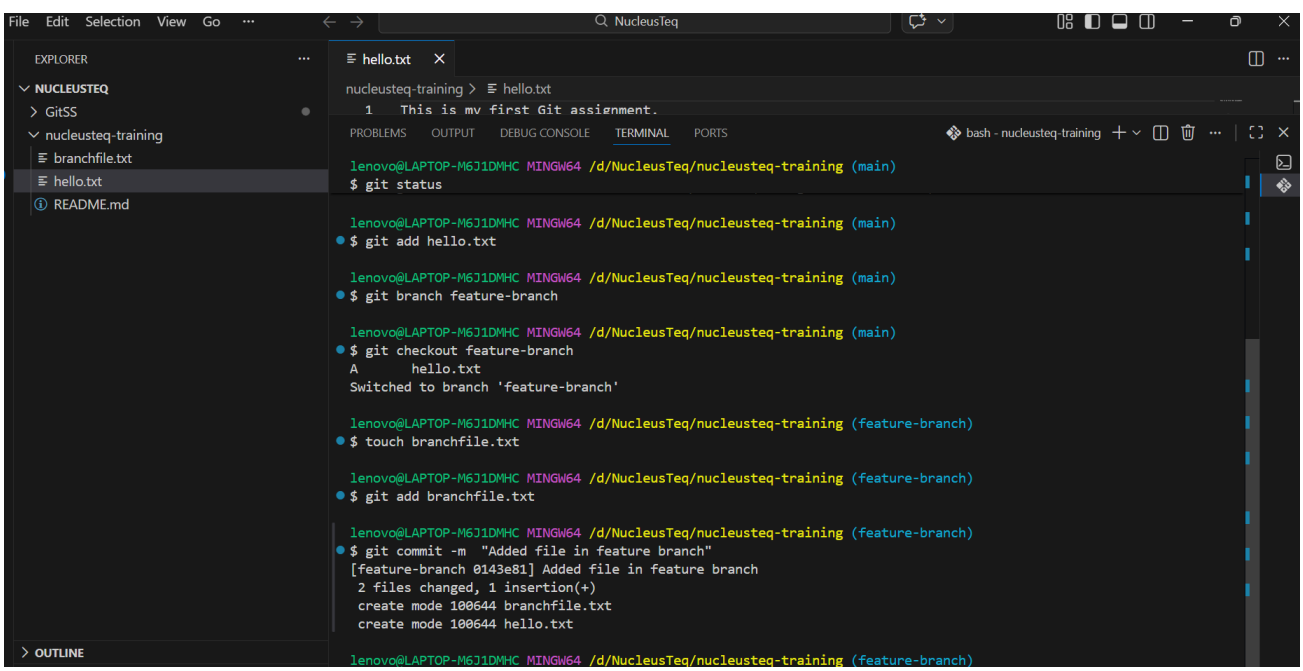


Step 4: Make Changes in the New Branch

While inside feature-branch, I created a new file called branchfile.txt and committed it. This shows how changes can be made in a branch without affecting the main codebase.

Commands used:

```
touch branchfile.txt
git add branchfile.txt
git commit -m "Added file in feature branch"
```

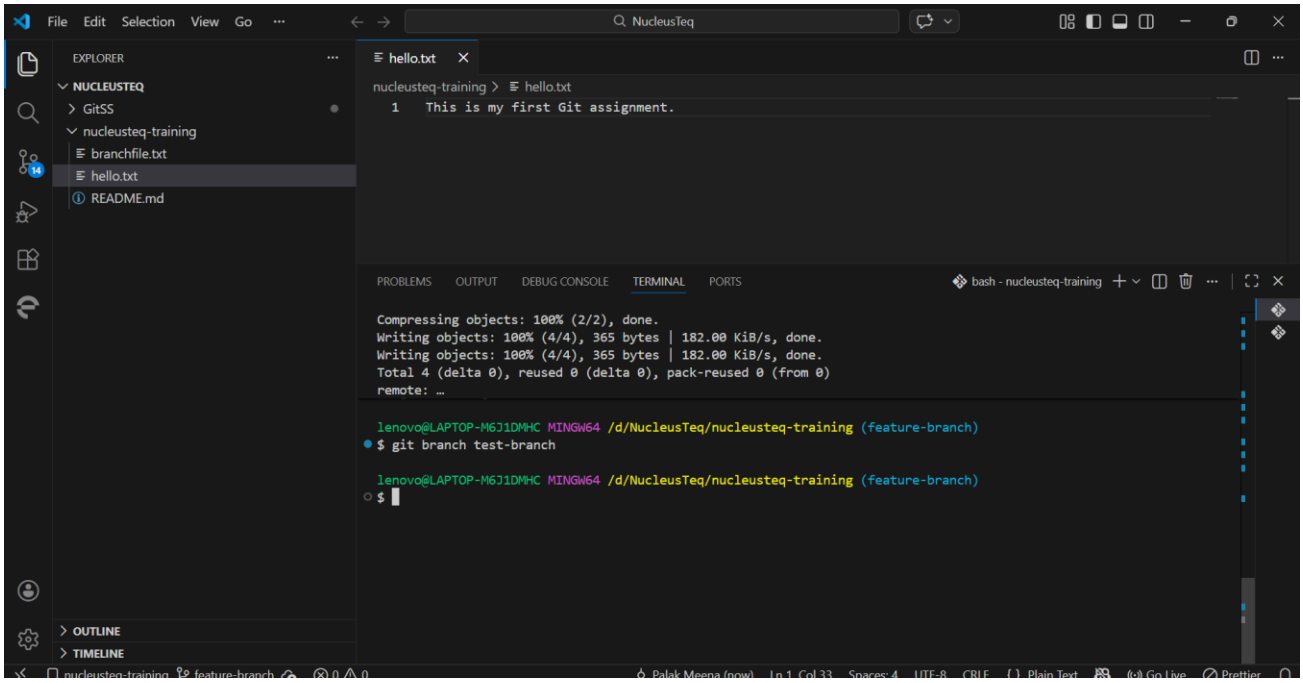


Step 5: Create Another Branch

Just to try it out, I also created one more branch called test-branch. This was mainly for experimenting and understanding how multiple branches work at the same time.

Command used:

```
git branch test-branch
```



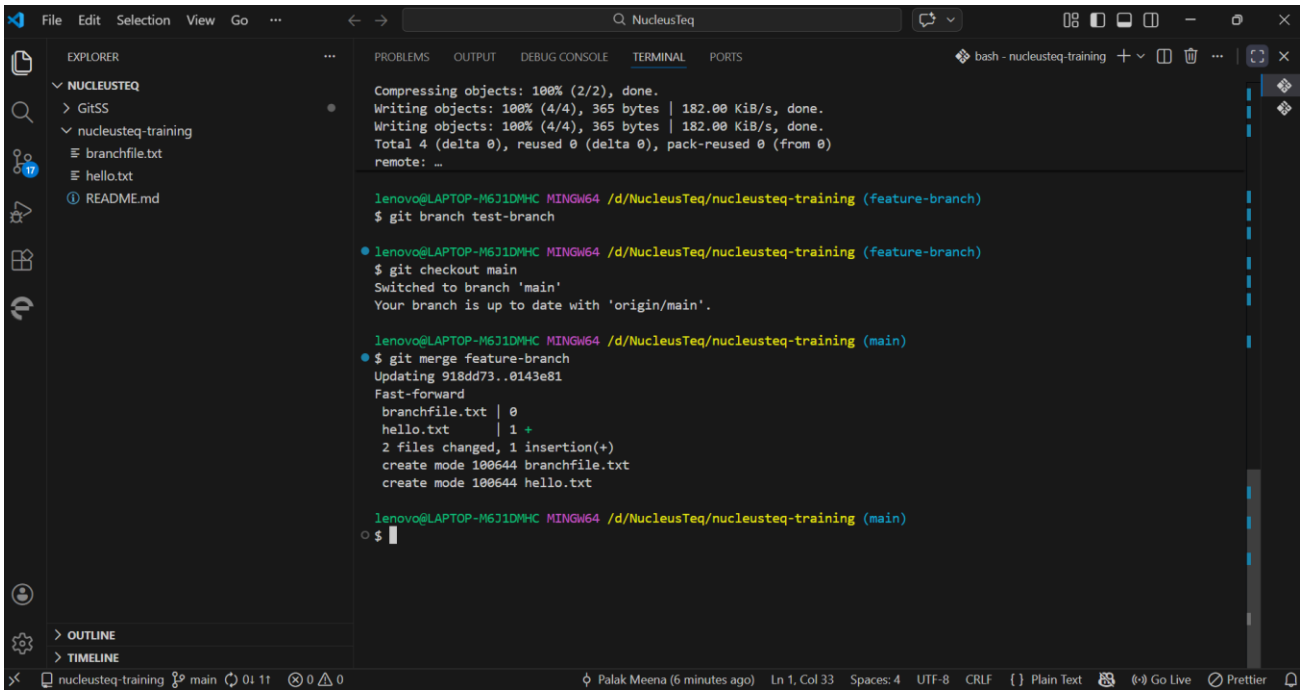
Step 6: Merge Branches

Once the changes in feature-branch were ready, I merged them into main. This is how developers bring their work back into the main codebase after completing a feature.

Commands used:

```
git checkout main
git merge feature-branch
```

First I switched back to the main branch, and then ran the merge command. This brought all the changes from feature-branch into main.



Step 7: Create .env File and Add to .gitignore

In real projects, some files contain sensitive data like passwords or API keys and should never be pushed to a remote repository. To practise this, I created a .env file and stored an email address in it.

Example content of .env file:

```
EMAIL=palakmeena14@gmail.com
```

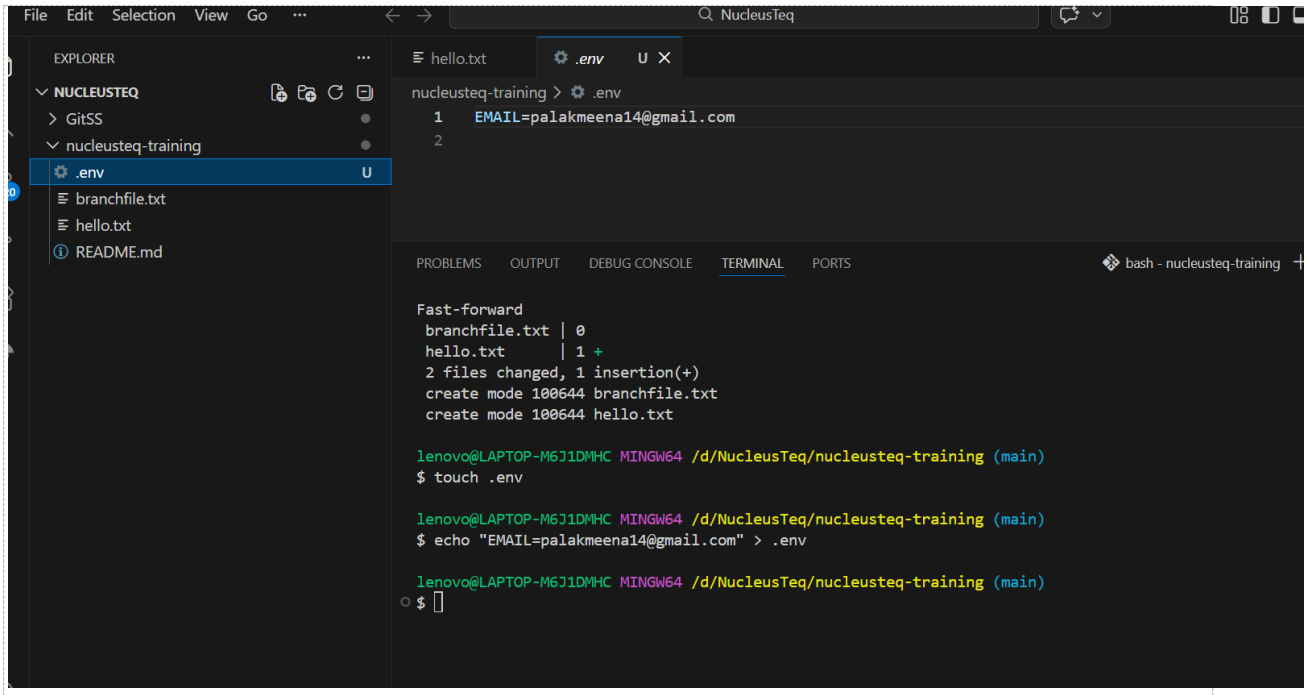
Then I created a .gitignore file and added .env to it so Git would ignore that file and never track it.

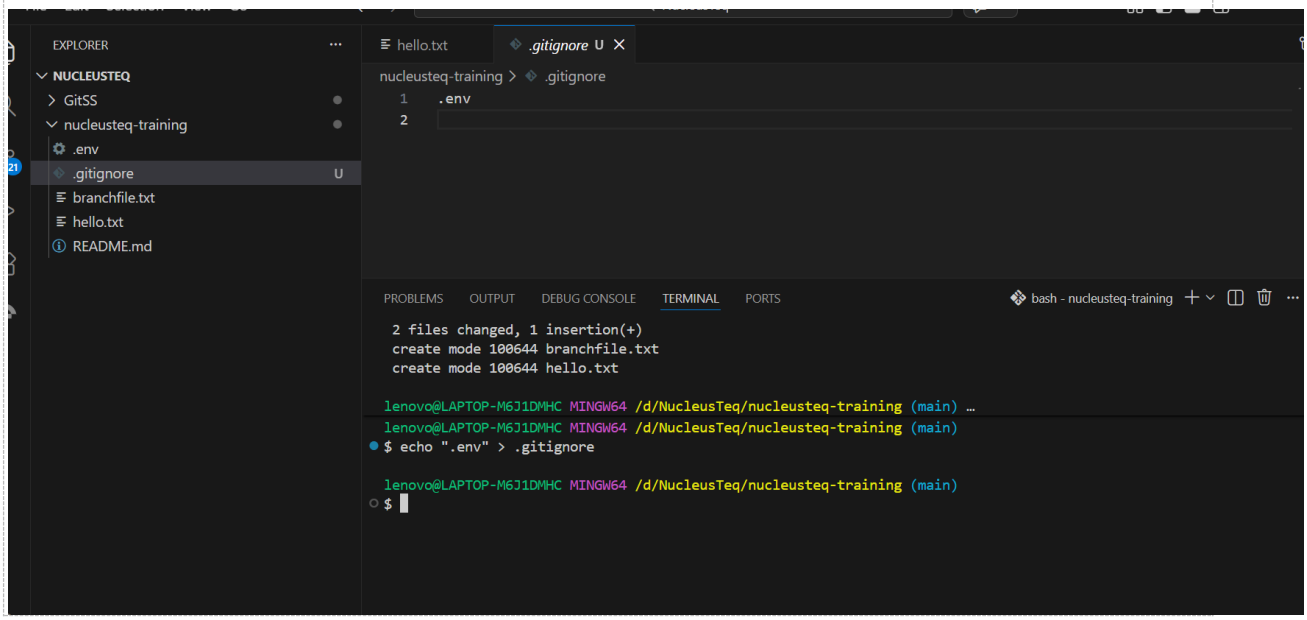
Commands used:

```
touch .env
touch .gitignore
echo "EMAIL=palakmeena14@gmail.com" > .env
echo ".env" > .gitignore
```

Content written inside .gitignore:

```
.env
```



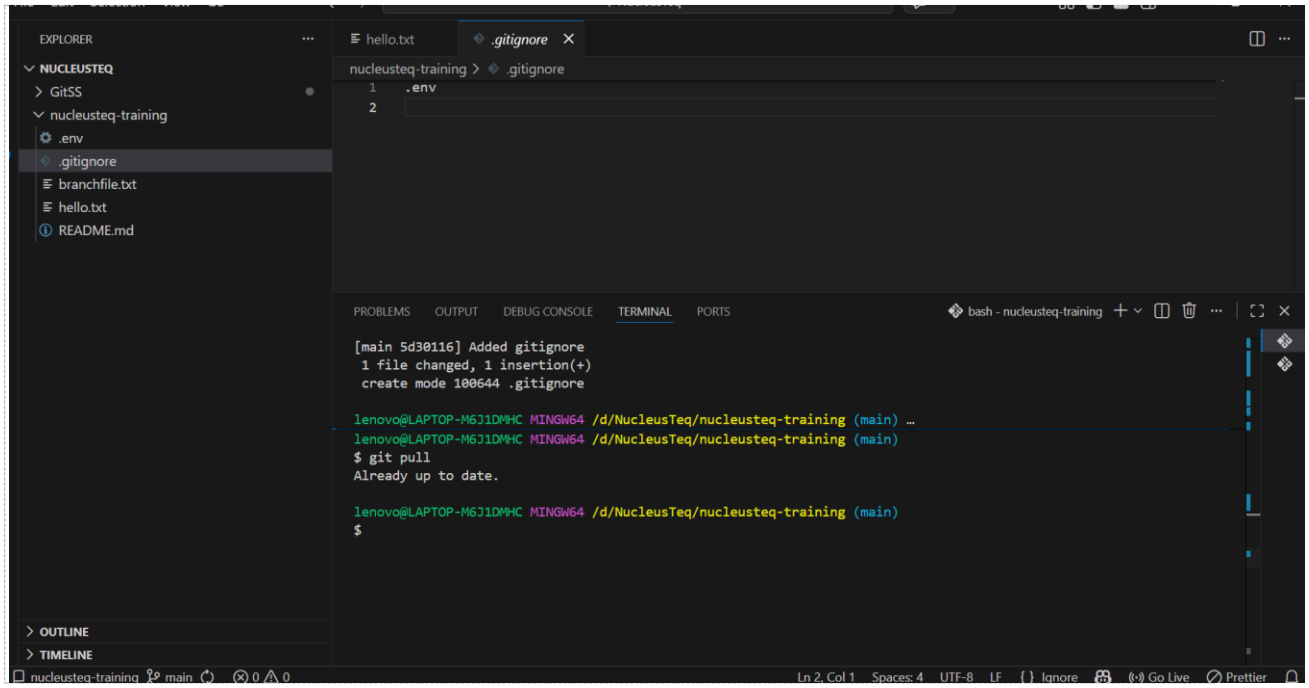


Step 8: Pull Changes from Remote Repository

To make sure my local repository is in sync with the remote one, I used the pull command. This is something you do regularly when working in a team, so you always have the latest version of the code.

Command used:

```
git pull
```

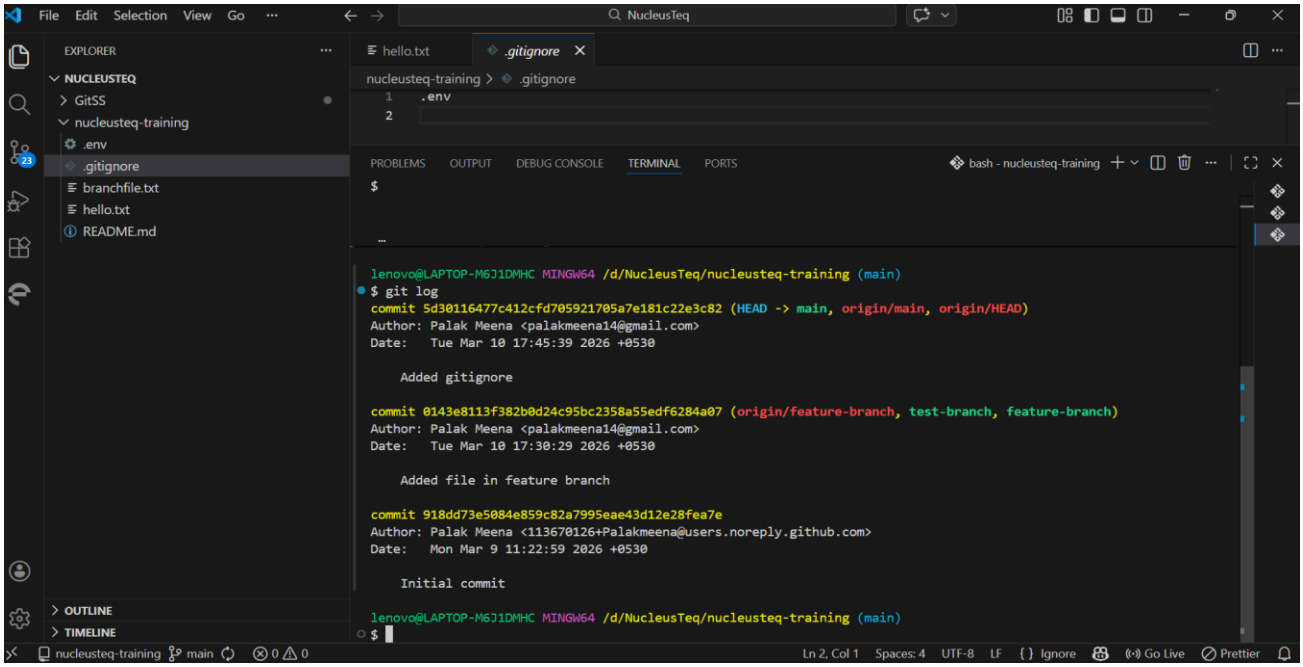


Step 9: View Git Log and History

To see the full commit history of the repository, I first used the git log command. This shows detailed information for each commit including the full commit ID, author name, date, and commit message.

Command used:

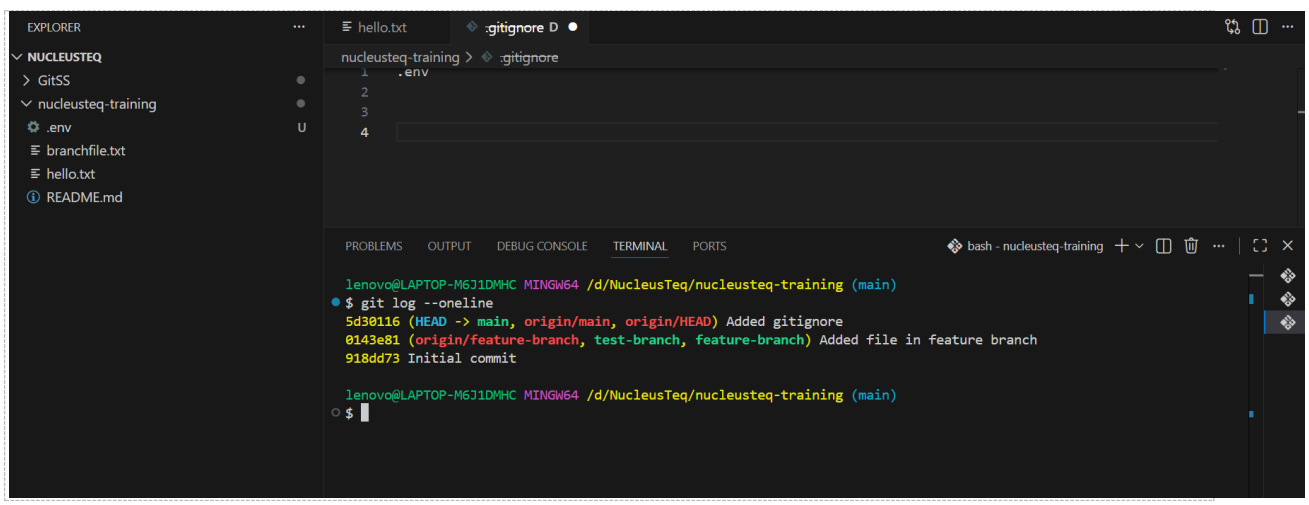
```
git log
```



I also used git log with the --oneline flag, which gives a much shorter and cleaner view. Each commit is shown on a single line with just the short commit ID and the message.

Command used:

```
git log --oneline
```



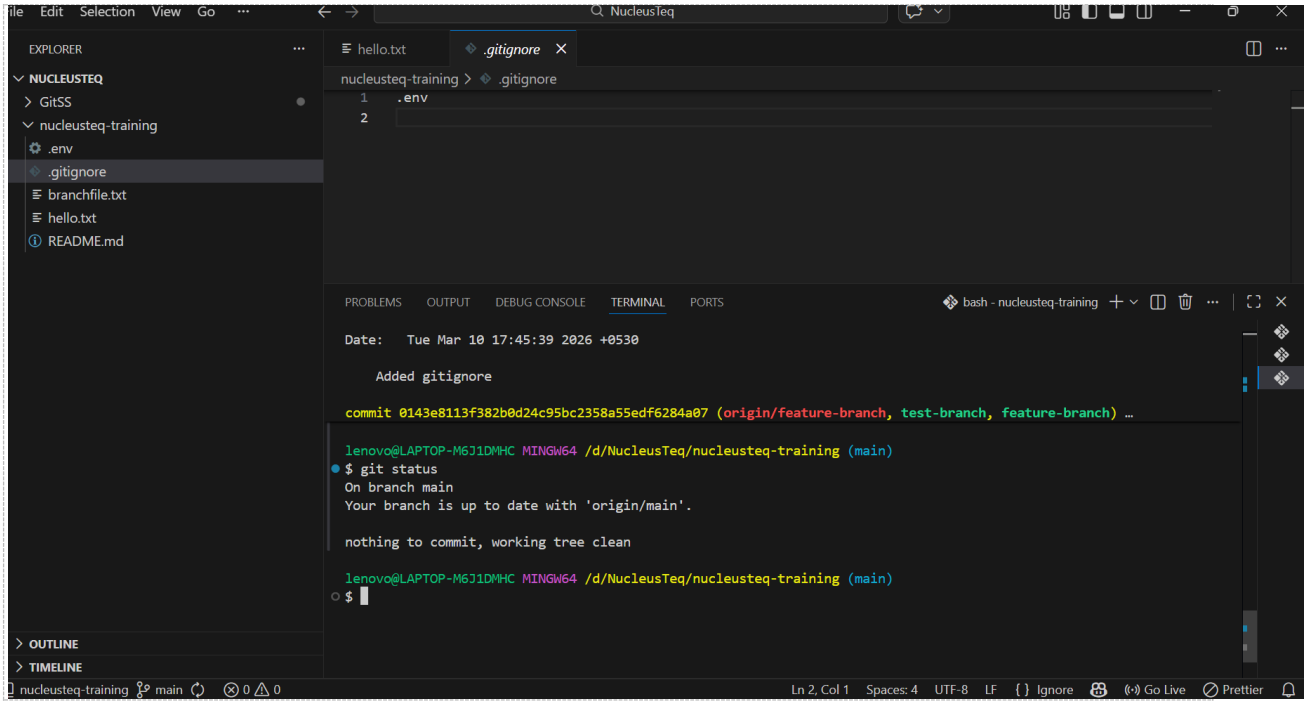
Step 10: Git Status and Staging

The git status command is something I used throughout the assignment to keep track of what was going on in the repository at any given point.

Command used:

```
git status
```

It tells you which files have been modified, which ones are staged and ready to be committed, and which ones are new and not yet tracked by Git.



Step 11: Revert and Reset

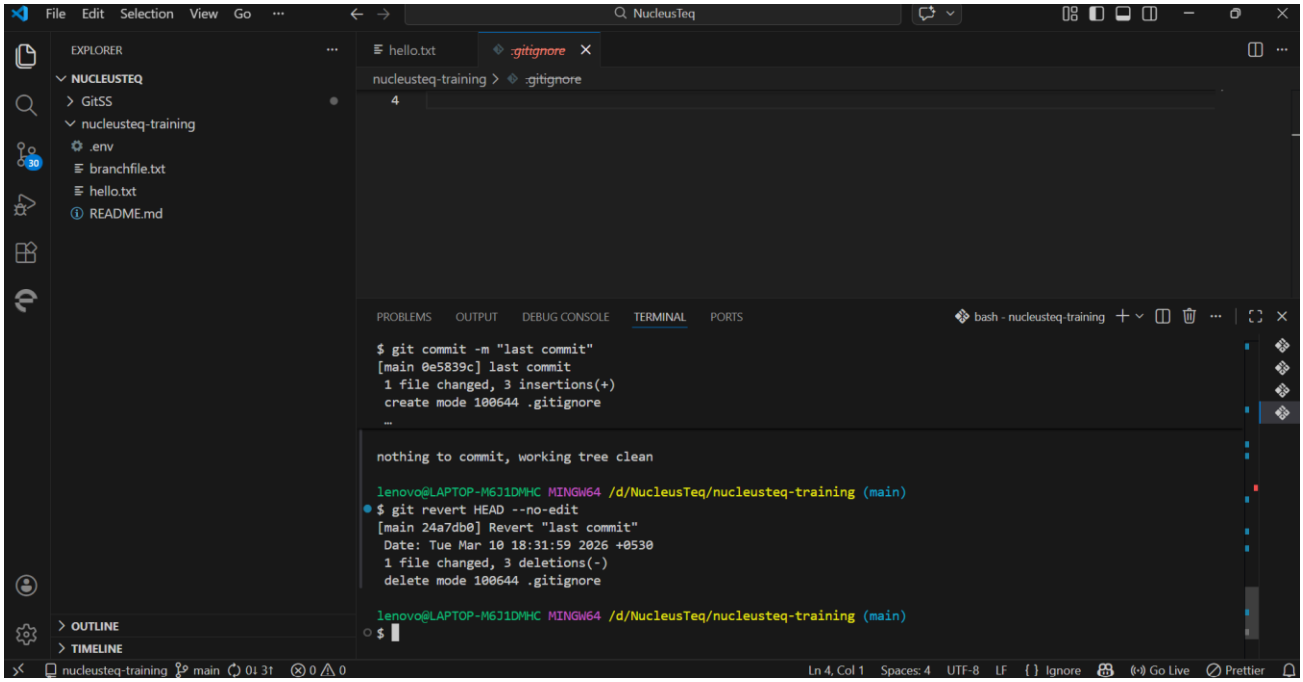
These two commands are used to undo changes, but they work in completely different ways.

git revert is the safer option it does not delete or change any existing commit. Instead it creates a brand new commit that reverses whatever was done in the last commit. The full history stays intact, which is why it is the preferred way to undo things in a shared or remote repository.

Command used:

```
git revert HEAD -no-edit
```

The --no-edit flag was added so that Git directly creates the revert commit without opening a text editor to ask for a commit message. Without this flag, Git opens the editor and waits for you to confirm the message before finishing the revert.

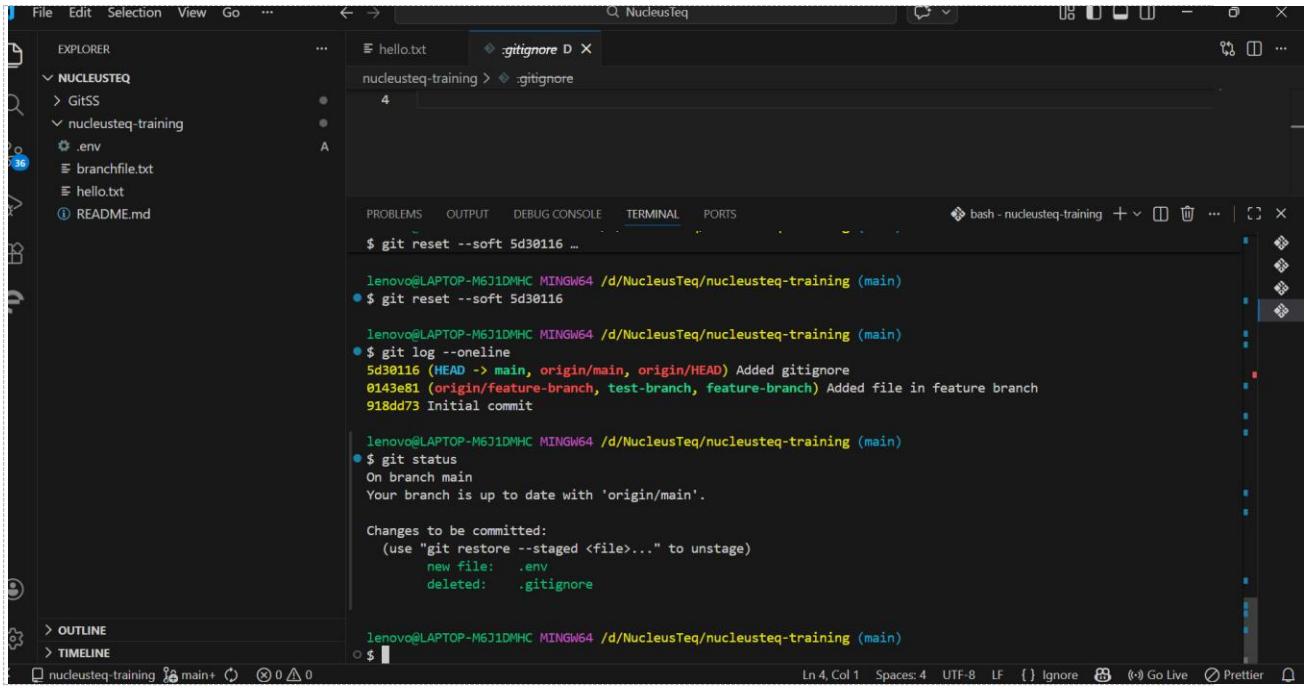


git reset on the other hand actually moves the branch pointer back to a specific earlier commit. Using --soft means the changes from the commits after that point are not lost, they are kept and moved back to the staging area, so you can recommit them if needed.

Command used:

```
git reset --soft 5d30116
```

Here 5d30116 is the short commit ID of the commit I wanted to go back to. This ID was taken from the git log --oneline output. After running this command, the branch was reset to that commit while all the changes remained staged.



Git Repository Link

Below is the link to my GitHub repository for this assignment:

https://github.com/username/nucleusteq-training

Conclusion

Doing this assignment gave me a much better understanding of how Git is actually used in day-to-day development work. Going through each step, setting up the repo, branching, merging, managing files with .gitignore, pulling from remote, and working with git log and git status, made things a lot clearer than just reading about them.

The revert and reset part was especially interesting because I got to see the difference between undoing commits safely versus actually moving back in history. Overall this was a good hands-on exercise and I feel more confident using Git going forward.